

When Recursion Helps: Evaluating RLM Against Direct LLM Prompting Across Seven Task Families Under Local Ollama Inference

Mayanka Gupta, Abhigyan Shekhar, Parth Bhat, Mayank Gupta, Anshul Desai, Anagha Avinash

Abstract

This paper presents a structured empirical comparison between two local execution paradigms over the same open-weight model: direct single-shot prompting (LLM) and agentic reasoning-loop execution via the RLM framework. Seven task families were evaluated: short reasoning, long-context retrieval, structured extraction, planning, PDF question answering, hallucination detection, and stochastic optimization. Performance was assessed primarily using latency and token usage, with per-run execution artifacts preserved for auditability. Across the matched tasks, direct prompting delivered lower latency and lower token cost in every benchmark pair. The largest local overheads appeared on the TSP reasoning tasks, where recursive execution required substantially more wall-clock time and tokens than the baseline. One stochastic adaptive TSP RLM run failed with exit code `-9`, indicating reliability stress under the most computationally demanding recursive workload. The benchmark suggests that on-device recursive execution should be treated as a specialized capability rather than a default deployment path when using a local `qwen2.5:7b` model through Ollama.

Index Terms—large language models, agentic systems, recursive execution, local inference, Ollama, benchmark evaluation

I INTRODUCTION

Direct prompting and agentic execution represent two materially different deployment strategies for large language models. In the direct path, a single prompt is issued to the model and the returned answer is treated as the final output. In the agentic path, the model is embedded inside a runtime that can iteratively reason, invoke tools, inspect intermediate states, and refine the final response across multiple steps. Although the second setup is often described as more powerful, the practical question is narrower: does that added machinery improve outcomes enough to justify its latency, token, and reliability costs?

This work addresses that question through a controlled benchmark comparison of direct local prompting and RLM-mediated execution across seven task families. The contribution is not a new reasoning architecture. Instead, the paper provides an execution-level comparison between a single-pass baseline and a recursive agentic runtime using saved repository logs as the evidence base. The objective is to identify how those trade-offs appear under a local Ollama deployment of `qwen2.5:7b`, where compute is limited and recursive overhead is

directly visible to the operator.

Unlike earlier hosted-model runs, the present paper is restricted to a single local batch:

- backend: `ollama`
- model: `qwen2.5:7b`
- paired runs per task: 5
- batch folder: `/Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama`

The emphasis of the paper is therefore operational. It asks whether recursive execution is practically worthwhile under local inference, and how much it costs relative to a plain LLM baseline over the same underlying model.

II LITERATURE REVIEW

A Recursive and Structured Reasoning

Prior work on recursive and tree-structured neural models established the broader intuition that hierarchical decomposition can improve representation quality when tasks are naturally compositional [1]. More recent inference-time reasoning methods such as Chain-of-Thought prompting, Tree-of-Thought search, and iterative self-refinement move that structure from latent representations into explicit intermediate reasoning steps [2, 3, 4]. Recent agentic frameworks extend the same principle to runtime control flow: instead of merely encoding structure in hidden states, they externalize structure as intermediate plans, sub-goals, tool calls, or spawned reasoning threads [6]. This lineage is directly relevant to RLM, whose promise is not stronger base-model knowledge, but better execution discipline on tasks that benefit from decomposition, checking, and iterative refinement.

B Agentic Memory and Long-Context Systems

Another strand of literature examines how language models cope with contexts that exceed the practical limits of a single forward pass. Systems such as MemGPT argue that long-context performance is often a systems problem rather than a pure architecture problem: the result depends on what is retained in working memory, what is externalized, and how retrieval is coordinated across steps [5]. These contributions are relevant because RLM, like other agentic runtimes, pays an execution tax in exchange for memory-management flexibility. The benchmark tasks in this study therefore test not only model intelligence, but also whether iterative orchestration helps or hurts performance relative to a direct full-context call.

C Long-Context Evaluation and Retrieval

Recent evaluation work has cautioned that many so-called long-context benchmarks are dominated by retrieval rather than by genuine multi-step synthesis

[7]. This distinction matters for the present comparison. If a task is principally a high-recall extraction problem, a direct prompt that places the full document in context may outperform a multi-step agent that repeatedly serializes, filters, or re-queries the same evidence. In such settings, additional planning becomes overhead rather than capability. By contrast, under-specified problems, ambiguous requirements, or tasks that reward explicit verification may benefit from iterative control logic. This framing helps explain why the direct LLM path wins several retrieval-heavy tasks in the benchmark while RLM’s clearest success appears on the hallucination benchmark.

D Reliability and Failure Surfaces

A recurring theme in the literature on tool-augmented and multi-step agents is that extra reasoning structure introduces both benefits and new failure modes. Intermediate traces improve observability, make refusals more principled, and create opportunities for self-correction. However, every additional loop, tool invocation, or decomposition step can introduce latency, state drift, formatting errors, and failure propagation from an early incorrect decision. The core empirical question for this benchmark is therefore not whether recursive reasoning is intrinsically superior, but under what conditions its additional control structure produces enough grounding benefit to justify its operational cost.

E Research Gap

Existing work provides strong conceptual arguments for structured reasoning, long-context memory management, and recursive computation, but it does not by itself answer a practical deployment question: when should a team prefer direct prompting over an agentic runtime for ordinary product workloads? Most prior studies evaluate specialized benchmarks or introduce new frameworks, rather than comparing single-shot and recursive execution on the same family of applied tasks with common logs, common metrics, and explicit attention to latency and token cost. The present benchmark addresses that gap.

III METHODOLOGY

A Experimental Design

The evaluation follows a controlled parallel-harness design. Each task is executed twice: once through a direct local model call and once through the RLM framework. Using the same underlying local model for both paths reduces model-capability confounds and isolates the effect of execution strategy. All raw outputs, wall-clock timings, and token usage were logged to markdown files under the benchmark batch directory.

B Task Selection

Task families were selected to cover the capability dimensions most relevant to agentic versus direct prompting comparisons. The suite spans short reasoning, long-context retrieval, structured extraction, planning, PDF question answering, hallucination detection, and stochastic optimization. Table 1 summarizes the task family, domain, and challenge type for each benchmark.

C System Configurations

LLM Path: A single prompt is issued to the local Ollama endpoint and the response is recorded without intermediate processing. This path serves as the local production baseline for latency- and cost-sensitive deployments.

RLM Path: The same prompt is executed through the RLM framework, which adds an iterative reasoning loop, REPL access, and tool invocation. Intermediate reasoning steps are observable, enabling self-correction across iterations before a final response is produced. Configuration parameters were held fixed within the local benchmark runner for the completed batch.

D Evaluation Metrics

This local paper emphasizes the metrics that were consistently available across the preserved run logs:

1. **Latency (seconds):** wall-clock time from invocation to final response.
2. **Token Usage:** total input plus output tokens per run.
3. **Exit Status:** whether a run completed cleanly.

Unlike the earlier hosted-model manuscript, this local paper does not present a new manually scored accuracy table, because the current focus is the reproducible local systems profile rather than a fresh human scoring pass.

E Scoring and Data Collection

Timings and token counts were extracted directly from saved markdown logs and normalized into:

- `raw_data.csv`
- `aggregate_metrics.csv`

The batch also includes a machine-generated `SUMMARY.md` and a local benchmark report:

- `/Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-qwen2.5-7`
- `/Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-qwen2.5-7`

F Task Suite

Table 1 summarizes the seven benchmark pairs used in the local run.

#	Task	Domain	Key Challenge
1	Battle of the Bastards	Short reasoning	Ally identification from fiction
2	AuthProxy Long Context	Retrieval	Five-question extraction from a noisy long document
3	Clinical Long Context	Structured extraction	Field, sentence, and conclusion extraction
4	Launch Note App	Planning	Structured product launch planning
5	PDF Cersei Warning	PDF QA	Quote and warning extraction from a document context
6	Under-specified TSP	Hallucination detection	Prompt omits the required distance matrix
7	Stochastic Adaptive TSP	Optimization	Expected-cost reasoning under a stochastic policy

IV RESULTS BY TASK

A Battle of the Bastards

This short reasoning task is the cheapest task in the local benchmark. The LLM path averaged 7.803 seconds and 313.6 total tokens. The RLM path averaged 108.379 seconds and 9,585.0 total tokens. The local result therefore shows a large recursive cost even on a short context.

B AuthProxy Long-Context Retrieval

The direct path averaged 23.689 seconds and 1,248.0 total tokens. The recursive path averaged 179.107 seconds and 13,000.6 total tokens. This is a clear long-context retrieval case where recursion substantially increased cost.

C Clinical Extraction

Clinical long-context extraction showed a similar but slightly narrower pattern: 104.282 seconds and 2,824.6 tokens for the LLM path versus 137.302 seconds

and 12,521.0 tokens for RLM. Even where wall-clock overhead was moderate relative to other tasks, token overhead remained large.

D Launch Note App Planning

The local planning benchmark averaged 91.281 seconds and 1,301.6 tokens for LLM versus 273.112 seconds and 13,111.0 tokens for RLM. This indicates that recursive orchestration incurred a large local systems cost for planning-style workloads.

E PDF Cersei Warning

The PDF-derived benchmark averaged 132.849 seconds and 5,008.4 tokens for LLM and 212.396 seconds and 20,396.0 tokens for RLM. Both paths completed in the local batch, but the recursive path was substantially more expensive.

F Under-specified TSP

The under-specified TSP benchmark produced one of the largest local cost gaps:

- LLM: 108.994s, 1,324.6 total tokens
- RLM: 510.680s, 23,515.2 total tokens

This task is especially useful for profiling the cost of recursive checking on hallucination-sensitive prompts.

G Stochastic Adaptive TSP

This was the most computationally demanding local benchmark. The LLM path averaged 164.309 seconds and 1,370.6 total tokens. The successful RLM runs averaged 843.710 seconds and 27,690.5 total tokens. One RLM run failed with exit code -9, leaving a success rate of 4/5.

V GRAPHICAL ANALYSIS

A Aggregate Performance Table

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
Battle of the Bastards	LLM	5/5 (100%)	7.803	7.756	275.0	38.6	313.6

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
Battle of the Bastards	RLM	5/5 (100%)	108.379	73.517	9145.0	440.0	9585.0
AuthProxy Long Context	LLM	5/5 (100%)	23.689	23.109	1082.0	166.0	1248.0
AuthProxy Long Context	RLM	5/5 (100%)	179.107	173.042	11749.0	1251.6	13000.6
Clinical Long Context	LLM	5/5 (100%)	104.282	111.993	1645.0	1179.6	2824.6
Clinical Long Context	RLM	5/5 (100%)	137.302	52.487	11503.8	1017.2	12521.0
Launch Note App	LLM	5/5 (100%)	91.281	81.166	257.0	1044.6	1301.6
Launch Note App	RLM	5/5 (100%)	273.112	276.742	10956.8	2154.2	13111.0
PDF Cersei Warning	LLM	5/5 (100%)	132.849	133.707	4096.0	912.4	5008.4
PDF Cersei Warning	RLM	5/5 (100%)	212.396	191.413	19444.4	951.6	20396.0
Under-specified TSP	LLM	5/5 (100%)	108.994	106.786	209.0	1115.6	1324.6
Under-specified TSP	RLM	5/5 (100%)	510.680	369.738	20109.2	3406.0	23515.2

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
Stochastic Adaptive TSP	LLM	5/5 (100%)	164.309	185.630	318.0	1052.6	1370.6
Stochastic Adaptive TSP	RLM	4/5 (80%)	843.710	700.832	24390.0	3300.5	27690.5

B Interpretation of Graphical Trends

The figures below visualize both the benchmark-level averages and the full run-by-run variation. Blue denotes LLM and red denotes RLM.

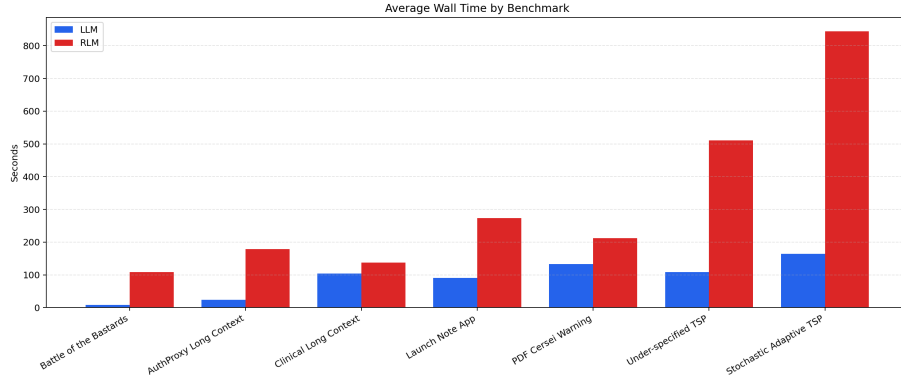


Figure 1: Average Wall Time by Benchmark

Figure 1. Average wall time by benchmark. The recursive path is slower in every benchmark pair, with the largest latency gap appearing in the stochastic adaptive TSP and under-specified TSP tasks.

Figure 2. Average total tokens by benchmark. The recursive path consumes substantially more tokens than the direct baseline across all seven benchmark families.

Figure 3. Per-run wall time. The recursive path shows large run-to-run variance on the harder tasks, especially the TSP benchmarks.

Figure 4. Per-run token usage. Token cost remains consistently low for the direct baseline and high for the recursive path, with one failed stochastic TSP recursive run appearing as a missing point.

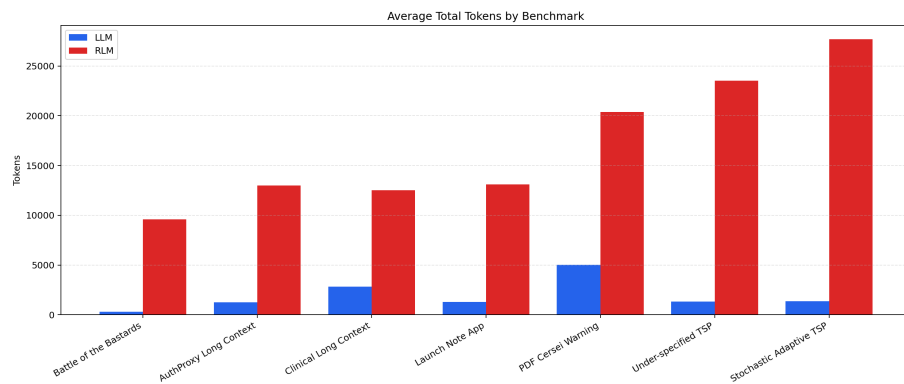


Figure 2: Average Total Tokens by Benchmark

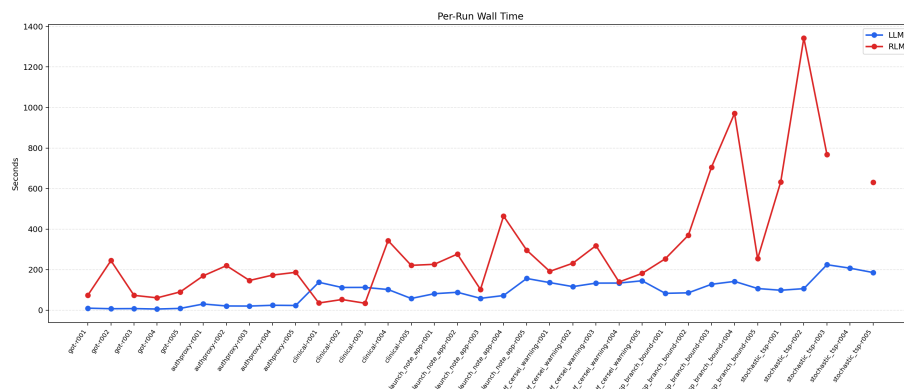


Figure 3: Per-Run Wall Time

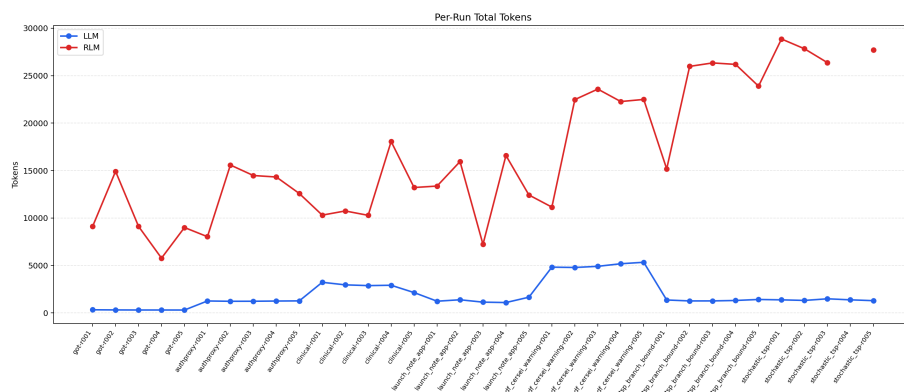


Figure 4: Per-Run Total Tokens

Taken together, the table and figures reveal the central trend of the local benchmark:

1. RLM is slower than LLM on every benchmark pair.
2. RLM uses more tokens than LLM on every benchmark pair.
3. The relative overhead becomes largest on the TSP-style reasoning tasks.
4. The only observed local run failure occurred in the hardest recursive benchmark.

These patterns suggest that local recursive execution is most costly precisely where the reasoning loops are deepest and the working context expands most rapidly.

VI RAW BENCHMARK DATA

The full raw data are stored in:

- /Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-qwen2.5-7
- /Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-qwen2.5-7

For readability, the main paper uses aggregate tables and figures, while the full run-by-run tables are included in Appendix A using compact per-benchmark layouts.

VII CROSS-TASK FINDINGS

Several patterns hold across the local benchmark.

First, direct prompting is more latency-efficient on every task. The smallest absolute local gap appears on Battle of the Bastards, but even there RLM is still far slower than the baseline. Second, direct prompting is more token-efficient on every task. Third, the recursive penalty is strongest on the TSP-style reasoning tasks. Fourth, recursive execution showed the only local run failure in the completed batch.

A Mechanistic Interpretation

RLM imposes cost because it externalizes intermediate reasoning. Each extra loop, code execution step, or intermediate trace adds tokens and latency. On a local model served through Ollama, these costs are not abstract accounting artifacts; they are directly reflected in wall-clock delay and machine utilization. The local results therefore make the systems tax of recursion visible in a way that can be obscured by faster hosted inference backends.

B Operational Interpretation

For local deployment, the benchmark suggests a simple default rule:

- use plain LLM execution when the task is a direct retrieval, extraction, or planning workload

- reserve RLM for situations where iterative checking, explicit traceability, or decomposition is operationally necessary

This paper does not argue that RLM is categorically inferior. It argues that, under this local open-weight setup, recursive execution is expensive enough that it should be invoked selectively.

VIII LIMITATIONS

This paper has several limitations.

1. It uses only one local model, `qwen2.5:7b`.
2. It measures a local Ollama deployment rather than a hosted inference provider.
3. It focuses on latency, tokens, and completion status, not on a new human-scored accuracy audit.
4. The sample size is five paired runs per task.
5. One recursive run failed on stochastic adaptive TSP, which slightly reduces the comparability of that pair.

These constraints mean the present manuscript should be read as a local systems benchmark rather than a final general statement about all LLM versus RLM deployments.

IX CONCLUSION

This IEEE-style local benchmark paper compared direct LLM execution against recursive RLM execution over seven benchmark families using Ollama with `qwen2.5:7b`. Across every benchmark pair, RLM incurred substantially more latency and token usage than the paired baseline. The difference was moderate on some long-context tasks and very large on the TSP-style reasoning tasks. One stochastic adaptive TSP recursive run failed outright. The central finding is therefore operational: under local open-weight inference, recursive execution is costly and should be used selectively. The raw markdown logs and extracted CSV tables preserved in the repository make the benchmark auditable and provide a foundation for a future scoring-focused follow-up study.

X FUTURE SCOPE

Several immediate extensions would strengthen this benchmark.

1. **Formal accuracy scoring.** The raw markdown logs now preserve enough evidence to support a second-pass scoring study with an explicit rubric and inter-rater agreement.
2. **Larger local model variants.** Repeating the same benchmark on stronger local models would clarify how much of the current overhead is due to recursion itself and how much is due to the limited speed and reasoning strength of `qwen2.5:7b`.

3. **Hosted versus local comparison.** A matched study using the same task suite under both Ollama and a hosted provider would help separate execution-strategy effects from inference-stack effects.
4. **Accuracy-cost frontier analysis.** With a clean human scoring layer, the benchmark could directly analyze whether any measured quality gains justify the additional recursive cost on specific task families.
5. **Failure analysis of recursive traces.** The preserved RLM logs enable a deeper study of where recursive execution fails: formatting drift, loop behavior, sub-call misuse, or simple model weakness.

Appendix A. Full Run Tables

A1. Battle of the Bastards

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	9.832	275	56	331
001	RLM	0	73.517	8816	299	9115
002	LLM	0	7.135	275	41	316
002	RLM	0	244.427	13982	936	14918
003	LLM	0	7.756	275	33	308
003	RLM	0	73.046	8753	367	9120
004	LLM	0	5.573	275	33	308
004	RLM	0	60.789	5483	289	5772
005	LLM	0	8.717	275	30	305
005	RLM	0	90.114	8691	309	9000

A2. AuthProxy Long Context

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	30.373	1082	178	1260
001	RLM	0	169.596	6619	1414	8033
002	LLM	0	20.656	1082	145	1227
002	RLM	0	219.642	13844	1741	15585
003	LLM	0	20.201	1082	149	1231
003	RLM	0	146.474	13488	990	14478
004	LLM	0	24.106	1082	171	1253
004	RLM	0	173.042	13385	937	14322
005	LLM	0	23.109	1082	187	1269
005	RLM	0	186.782	11409	1176	12585

A3. Clinical Long Context

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	137.570	1645	1583	3228
001	RLM	0	35.334	10181	125	10306
002	LLM	0	111.993	1645	1312	2957
002	RLM	0	52.487	10392	351	10743
003	LLM	0	112.244	1645	1234	2879
003	RLM	0	34.254	10160	123	10283
004	LLM	0	101.873	1645	1270	2915
004	RLM	0	343.306	15177	2883	18060
005	LLM	0	57.730	1645	499	2144
005	RLM	0	221.131	11609	1604	13213

A4. Launch Note App

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	81.166	257	978	1235
001	RLM	0	226.218	11223	2144	13367
002	LLM	0	88.222	257	1129	1386
002	RLM	0	276.742	13106	2843	15949
003	LLM	0	58.346	257	879	1136
003	RLM	0	102.077	5808	1416	7224
004	LLM	0	72.258	257	837	1094
004	RLM	0	464.365	14161	2426	16587
005	LLM	0	156.415	257	1400	1657
005	RLM	0	296.160	10486	1942	12428

A5. PDF Cersei Warning

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	135.947	4096	728	4824
001	RLM	0	191.413	10247	903	11150
002	LLM	0	116.163	4096	692	4788
002	RLM	0	231.660	21579	899	22478
003	LLM	0	133.442	4096	816	4912
003	RLM	0	318.367	22295	1289	23584
004	LLM	0	133.707	4096	1085	5181
004	RLM	0	139.048	21496	777	22273
005	LLM	0	144.988	4096	1241	5337
005	RLM	0	181.494	21605	890	22495

A6. Under-specified TSP

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	83.021	209	1154	1363
001	RLM	0	253.519	12514	2661	15175
002	LLM	0	85.607	209	1055	1264
002	RLM	0	369.738	23510	2468	25978
003	LLM	0	127.640	209	1057	1266
003	RLM	0	705.174	22245	4097	26342
004	LLM	0	141.916	209	1103	1312
004	RLM	0	970.961	20689	5498	26187
005	LLM	0	106.786	209	1209	1418
005	RLM	0	254.008	21588	2306	23894

A7. Stochastic Adaptive TSP

Run	Mode	Exit	Wall (s)	Input	Output	Total
001	LLM	0	98.209	318	1059	1377
001	RLM	0	633.152	24446	4417	28863
002	LLM	0	106.263	318	999	1317
002	RLM	0	1342.957	24535	3297	27832
003	LLM	0	224.292	318	1170	1488
003	RLM	0	768.512	23597	2772	26369
004	LLM	0	207.149	318	1061	1379
004	RLM	-9	-	-	-	-
005	LLM	0	185.630	318	974	1292
005	RLM	0	630.217	24982	2716	27698

References

- [1] K. S. Tai, R. Socher, and C. D. Manning, “Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks,” in Proc. ACL, 2015.
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Advances in Neural Information Processing Systems, 2022.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” in Advances in Neural Information Processing Systems, 2023.
- [4] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” in Advances in Neural Information Processing Systems, 2023.
- [5] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” 2023, arXiv:2310.08560.

- [6] P. Schroeder, N. Morgan, H. Luo, and J. Glass, “THREAD: Thinking Deeper with Recursive Spawning,” 2024.
- [7] O. Goldman, A. Jacovi, A. Slobodkin, A. Maimon, I. Dagan, and R. Tsarfaty, “Is It Really Long Context if All You Need Is Retrieval? Towards Gen- uinely Difficult Long Context NLP,” 2024.